

Example: Vending Machine Controller in Verilog – Full Model

```
module Vending (output reg Dispense, Change,
                input  wire Clock, Reset, D_in, Q_in);

reg [1:0] current_state, next_state;
parameter sWait = 2'b00, s25 = 2'b01, s50 = 2'b10;

always @ (posedge Clock or negedge Reset)
begin: STATE_MEMORY
    if (!Reset)
        current_state <= sWait;
    else
        current_state <= next_state;
    end

always @ (current_state or D_in or Q_in)
begin: NEXT_STATE_LOGIC
    case (current_state)
        sWait      : if (Q_in == 1'b1)
                        next_state = s25;
                      else
                        next_state = sWait;
        s25        : if (Q_in == 1'b1)
                        next_state = s50;
                      else
                        next_state = s25;
        s50        : if (Q_in == 1'b1)
                        next_state = sWait;
                      else
                        next_state = s50;
        default    : next_state = sWait;
    endcase
end

always @ (current_state or D_in or Q_in)
begin: OUTPUT_LOGIC
    case (current_state)
        sWait      : if (D_in == 1'b1)
                        begin
                            Dispense = 1'b1; Change = 1'b1;
                        end
                      else
                        begin
                            Dispense = 1'b0; Change = 1'b0;
                        end
        s25        : begin
                        Dispense = 1'b0; Change = 1'b0;
                    end
        s50        : if (Q_in == 1'b1)
                        begin
                            Dispense = 1'b1; Change = 1'b0;
                        end
                      else
                        begin
                            Dispense = 1'b0; Change = 1'b0;
                        end
        default    : begin
                        Dispense = 1'b0; Change = 1'b0;
                    end
    endcase
end
endmodule
```

State variables and state encoding.

State memory block.

Next state logic block.

Output logic block.

Example: 2-Bit Up/Down Counter in Verilog – Full Model (Three Block Approach)

```
module Counter_2bit_UpDown (output reg [1:0] CNT,  
                           input wire      Clock, Reset, Up);
```

```
    reg [1:0] current_state, next_state;  
    parameter C0 = 2'b00,  
              C1 = 2'b01,  
              C2 = 2'b10,  
              C3 = 2'b11;
```

← State variables and
state encoding.

```
always @ (posedge Clock or negedge Reset)  
begin: STATE_MEMORY  
    if (!Reset)  
        current_state <= C0;  
    else  
        current_state <= next_state;  
end
```

← State memory block.

```
always @ (current_state or Up)  
begin: NEXT_STATE_LOGIC  
    case (current_state)  
        C0      : if (Up == 1'b1) next_state = C1; else next_state = C3;  
        C1      : if (Up == 1'b1) next_state = C2; else next_state = C0;  
        C2      : if (Up == 1'b1) next_state = C3; else next_state = C1;  
        C3      : if (Up == 1'b1) next_state = C0; else next_state = C2;  
        default : next_state = C0;  
    endcase  
end
```

← Next state logic block.

```
always @ (current_state)  
begin: OUTPUT_LOGIC  
    case (current_state)  
        C0      : CNT = 2'b00;  
        C1      : CNT = 2'b01;  
        C2      : CNT = 2'b10;  
        C3      : CNT = 2'b11;  
        default : CNT = 2'b00;  
    endcase  
end
```

← Output logic block. Note that since this is a
Moore machine only the current state is listed
in the sensitivity list.

```
endmodule
```

BLOCK DESIGN - design_1

Sources

x

Design

Signals



0

Design Sources (3)

Verilog Header (1)

design_1_wrapper (design_1_wrapper.v) (1)

ad_ip_jesd204_tpl_dac (ad_ip_jesd204_tpl_dac.v) (2)

i_regmap : ad_ip_jesd204_tpl_dac_regmap (ad_ip_jesd204_tpl_dac.v) (1)

i_up_axi : xil_defaultlib.up_axi

i_up_dac_common : xil_defaultlib.up_dac_common

i_up_tpl_dac : xil_defaultlib.up_tpl_common

g_channel[0].i_up_dac_channel : xil_defaultlib.up_dac_channel

g_channel[1].i_up_dac_channel : xil_defaultlib.up_dac_channel

i_core : xil_defaultlib.ad_ip_jesd204_tpl_dac_core

Constraints

constrs_1

Simulation Sources (3)

sim_1 (3)

Verilog Header (1)

design_1_wrapper (design_1_wrapper.v) (1)

ad_ip_jesd204_tpl_dac (ad_ip_jesd204_tpl_dac.v) (2)

Utility Sources



赛灵思中文社区论坛欢迎您 (Archived) – [marxi](#) (Customer) asked a question.
2020年12月17日 at 01:57

第一次用vivado，请问如何层次化添加子模块

在写一个CPU的项目，顶层设计要把子模块添加进来。但是在CPU顶层项目里添加子文件的话会搞到同一级里，就像这样



想问下怎么才能弄成下面这种树形的



以前用的quartus，第一次用vivado，不太懂这种层次级或者说调用子模块的情况下，子模块的文件怎么用这种树形的方式添加进来，希望有经验的同志给点拨一下，谢谢

开发工具

Like Answer Share

2 answers · 830 views

Top Rated Answers



[graces](#) (Employee)

3 years ago

****BEST SOLUTION****

顶层RTL文件里如果例化了子模块，例如CPU.v里面例化了ALU模块，添加文件后source窗口会自动更新成对应的树形结构。

Selected as Best · Like

All Answers



[graces](#) (Employee)

3 years ago

****BEST SOLUTION****

顶层RTL文件里如果例化了子模块，例如CPU.v里面例化了ALU模块，添加文件后source窗口会自动更新成对应的树形结构。

Selected as Best · Like · Reply



[marxi](#) (Customer)

3 years ago

谢谢，已经成功了

Like · Reply

Log In to Answer

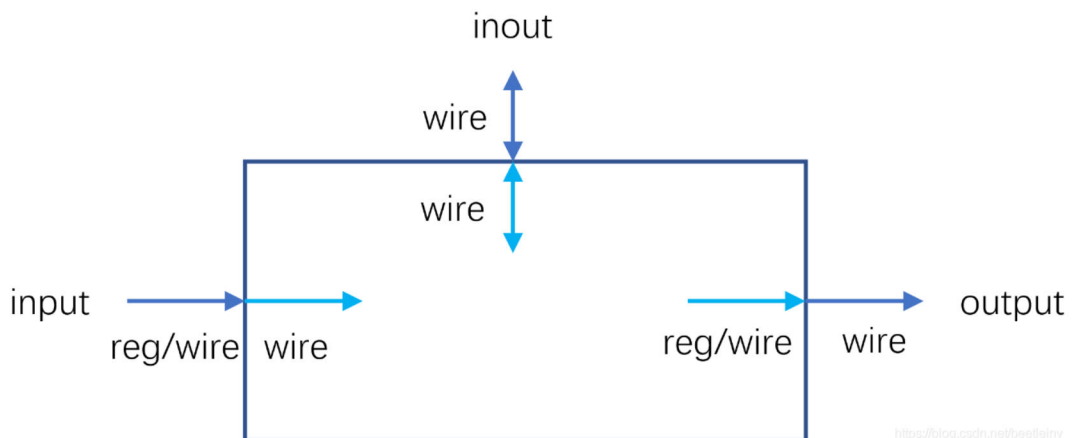
verilog实例化时端口信号传递规则

 blog.csdn.net/beetleinv/article/details/108798692

```
module hello_top(  
    input clk_t,  
    input rst_t,  
    input rxd_t,  
    output txd_t  
);  
  
    uart_send u_uart_send(  
        .clk (clk_t),  
        .rst (rst_t),  
        .txd (txd_t)  
    );  
  
endmodule
```

```
module uart_send(  
    input clk,  
    input rst,  
    output txd  
);  
  
    // coding from here  
  
endmodule
```

示图



表格

端口	从模块内部看	从模块外部看
input 输入端口	必须为线网类型	额可以线网类型或寄存器类型
output 输出端口	可以是线网类型或寄存器类型	必须为线网类型
inout 输入输出端口	必须为线网类型	必须为线网类型

说明

端口连接规则 将一个端口看成由相互链接的两个部分组成，一部分位于模块内部，另一部分位于模块外部。当在一个模块中调用（实例引用）另一个模块时，端口之间的连接必须遵守一些规则。

- 1、输入端口：从模块内部来讲，输入端口必须为线网数据类型，从模块外部来看，输入端口可以连接到线网或者reg数据类型的变量。
- 2、输出端口：从模块内部来讲，输出端口可以是线网或者reg数据类型，从模块外部来看，输出必须连接到线网类型的变量（显式，隐式），而不能连接到reg类型的变量。
- 3、输入/输出端口（必须为wire）从模块内部来讲，输入/输出端口必须为线网数据类型；从模块外部来看，输入/输出端口也必须连接到线网类型的变量。

Verilog中 reg和wire 用法和区别以及always和assign的区别

 blog.csdn.net/u012158332/article/details/80965063

1、从仿真角度来说，HDL语言面对的是编译器，相当于使用软件思路，此时：

wire对应于连续赋值，如assign；

reg对应于过程赋值，如always，initial；

2、从综合角度，HDL语言面对的是综合器，相当于从电路角度来思考，此时：

wire型变量综合出来一般情况下是一根导线。

reg变量在always中有两种情况：

(1) always @ (a or b or c) 形式的，即不带时钟边沿的，综合出来还是组合逻辑；

(2) always @ (posedge clk) 形式的，即带有边沿的，综合出来一般是时序逻辑，会包含触发器 (Flip-Flop)

3、设计中，输入信号一般来说不能判断出上一级是寄存器输出还是组合逻辑输出，对于本级来说，就当成一根导线，即wire型。而输出信号则由自己来决定是reg还是组合逻辑输出，wire和reg型都可以。但一般的，整个设计的外部输出（即最顶层模块的输出），要求是reg输出，这比较稳定、扇出能力好。

4、Verilog中何时要定义成wire型？

情况一：assign语句

例如：

```
reg    a,b;

wire   out;

.....

assign out = a & b;
```

如果把out定义成reg型，对不起，编译器报错！

情况二：元件实例化时必须用wire型

例如：

```
wire    dout;

ram    u_ram

(

    ....

    .out(dout);

)
```

wire为无逻辑连线，wire本身不带逻辑性，所以输入什么就输出什么。所以如果用always语句对wire变量赋值，对不起，编译器报错。

那么你可能会问，`assign c = a & b;` 不是对wire的赋值吗？

并非如此，综合时是将 `a & b` 综合成 `a`、`b` 经过一个与门，而`c`是连接到与门输出线，真正综合出来的是与门&，不是`c`。

5、何时用reg、何时用wire？

大体来说，reg和wire类似于C、C++的变量，但若此变量要放在begin...end之内，则该变量只能是reg型；在begin...end之外，则用wire型；

使用wire型时，必须搭配assign；reg型可以不用。

input、output、inout预设值都是wire型。

在Verilog中使用reg型，并不表示综合出来就是暂存器register：在组合电路中使用reg，组合后只是net；在时序电路中使用reg，合成后才以Flip-Flop形式表示的register触发器。

6、reg和wire的区别：

reg型数据保持最后一次的赋值，而wire型数据需要持续的驱动。wire用在连续赋值语句assign中；reg用于always过程赋值语句中。

在连续赋值语句assign中，表达式右侧的计算结果可以立即更新到表达式的左侧，可以理解为逻辑之后直接连接了一条线，这个逻辑对应于表达式的右侧，这条线对应于wire；

在过程赋值语句中，表达式右侧的计算结果在某种条件的触发下放到一个变量当中，这个变量可以声明成reg型，根据触发条件的不同，过程语句可以建模不同的硬件结构：

(1) 如果这个条件是时钟上升沿或下降沿，那硬件模型就是一个触发器，只有是指定了always@ (posedge or negedge) 才是触发器。

(2) 如果这个条件是某一信号的高低电平，那这个硬件模型就是一个锁存器。

(3) 如果这个条件是赋值语句右侧任意操作数的变化，那这个硬件模型就是一个组合逻辑。

7、过程赋值语句always@和连续赋值语句assign的区别：

(1) wire型用于assign的赋值，always@块下的信号用reg型。这里的reg并不是真正的触发器，只有敏感列表内的为上升沿或下降沿触发时才综合为触发器。

(2) 另一个区别，举例：

```
wire    a;

reg     b;

assign  a = 1'b0;

always@(*)

    b = 1'b0;
```

上面例子仿真时a将会是0，但是b的状态是不确定的。因为Verilog规定，always@(*)中的*指的是该always块内的所有输入信号的变化为敏感列表，就是说只有当always@(*)块内输入信号发生变化，该块内描述的信号才会发生变化。

像always@(*) b= 1'b0; 中由于1'b0是个常数，一直没有变化，由于b的足组合逻辑输出，所有复位时没有明确的值--即不确定状态，又因为always@(*)块内没有敏感信号变化，此时b信号一直保持不变，即不确定是啥，取决于b的初始状态。

在Verilog中如何给端口选择正确的数据类型

 blog.csdn.net/qq_27386569/article/details/103123429

版权声明：本文为博主原创文章，遵循 [CC 4.0 BY-SA](#) 版权协议，转载请附上原文出处链接和本声明。

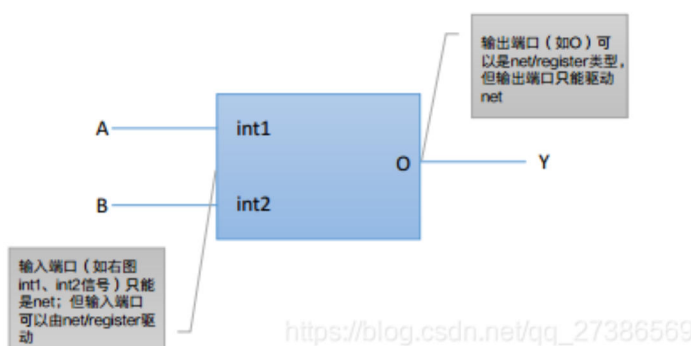
本文链接：https://blog.csdn.net/qq_27386569/article/details/103123429

1 篇文章 0 订阅

订阅专栏

今天要介绍的知识点相信很多同学都已经很清楚了，在这里做介绍只是对我和一些新手做一下知识的巩固。

那么到底如何理解输入和输出端口到底是使用reg类型还是wire类型？



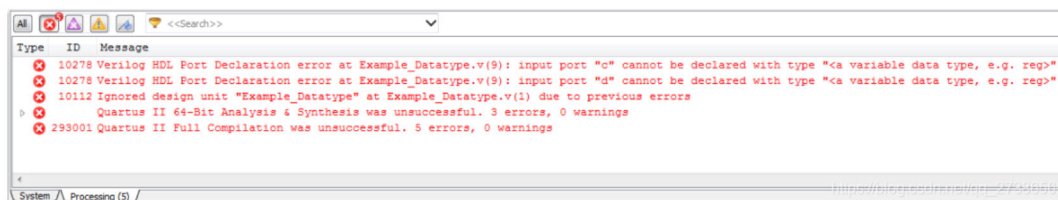
如图1所示，对于输入A和输入B来说，我们只能使用线网类型，但是，输入A和输入B这两个端口可以由寄存器和线网所驱动。也就是说，寄存器和线网可以连接到这两个输入端口作为输入源。对于输出端口Y来讲，可以是线网，也可以是寄存器类型，但是，对于输出端口Y，它只能驱动线网类型。这样说比较抽象，下面我们结合具体的实例进行说明，代码如下：

```
module Datatype_eg(
a,b,c,d,o1,o2
);
input a,b,c,d;
ouput o1,o2;

reg c,d;
reg o1,o2;
assign o2=c&& d;

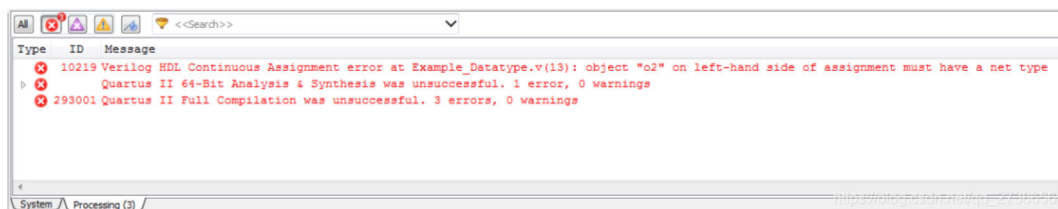
always@(a or b)
begin
    if(a)
        o1= b;
    else
        o1=1'b0;
end
endmodule
```

在Quartus II上对上述代码进行编译，会显示出以下错误，如图2所示：



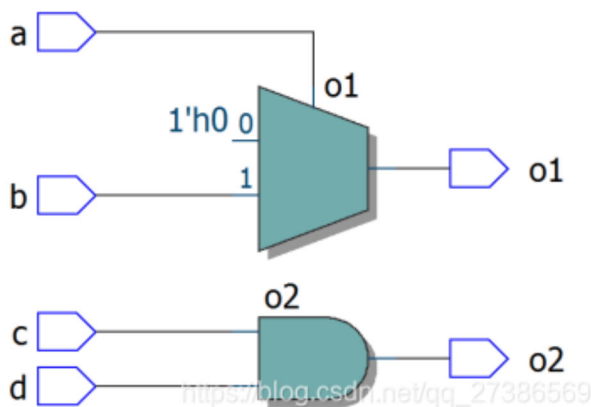
从图2中可知，c不能被声明为reg类型，也就是错误提示中的variable data type,这里为什么会是variable,原因是在新的Verilog-2001标准中，已经没有寄存器类型（register）类型，它们把register改成了variable变量类型，但是一般来说，我们还是比较习惯称为寄存器类型。

言归正传，为什么c和d不能被定义为寄存器类型，是因为c和d是输入端口，前面已经说过了，输入端口不能定义为寄存器类型，因此c和d只能定义为wire类型。这里进行了代码修改，将reg c和reg d注释掉就可以了，默认的类型就是wire类型。修改完成后，代码仍有错误，如图3所示：



如图3所示，o2必须为线网类型，也许有同学会问，前面说对于输出端口来说，可以是线网，也可以是寄存器类型，为什么在这里输出端口o2必须为线网类型不能为寄存器类型呢？这是因为o2是由关键字assign（连续赋值语句）指定的，必须是wire类型，这一点大家可以记住。那么对于o1必须定义为reg类型的，是由于它是在always块里赋值的。因此，只需要将o2注释掉，编译就没有错误了。

这里再补充一点：



如图4所示，尽管我们将o1声明成了reg类型，但是并没有综合成触发器。原因是对于寄存器reg类型，在always等过程中被赋值的信号，如果该always模块中描述的是时序逻辑电路，那么该

信号常常被综合为D触发器，如果该always模块中描述的是组合逻辑电路，那么该信号会被综合成连线。

verilog基础教程

FPGA参数定义 reg&wire 详解_fpge定义参数_朴实姐己的博客-CSDN博客

blog.csdn.net/weixin_46188211/article/details/123344679

版权声明：本文为博主原创文章，遵循 [CC 4.0 BY-SA](#) 版权协议，转载请附上原文出处链接和本声明。

本文链接：https://blog.csdn.net/weixin_46188211/article/details/123344679

在写FPGA代码进行参数类型定义时，对于写reg还是wire常常叫人迷惑，下面我将分为以下三种情况，详细解释如何定义参数类型

目录

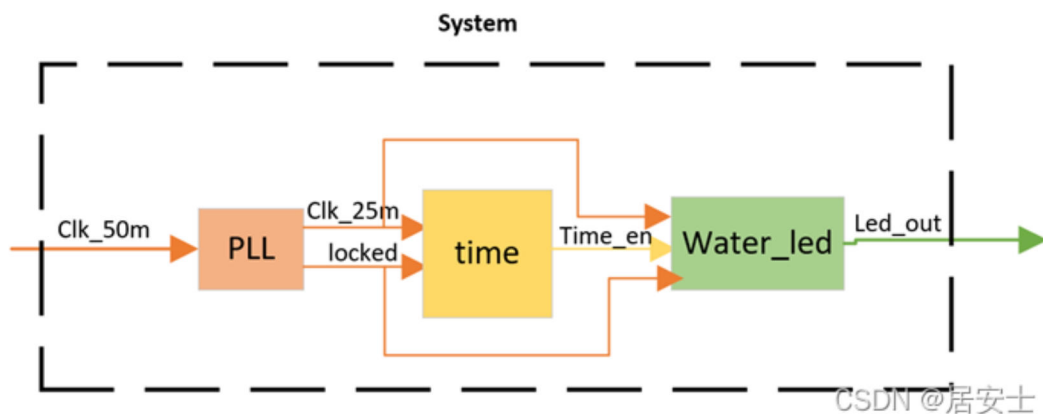
一、输入输出定义

二、例化定义

三、仿真定义

一、输入输出定义

输入输出定义是指，在写程序的时候，向各个程序模块里面输入和输出某些参数，例如下面的流水灯程序，PLL，time，water_led三个模块分别有不同的输入输出



针对这种情况应该这样定义

input 都为wire型；

output 若在always块下定义为reg型，写作output reg xxxxx

(不加reg默认wire，不用写output wire xxxx)

(原因：在设计中，输入信号一般来说你是不知道上一级是寄存器输出还是组合逻辑输出，那么对于本级来说就是一根导线，也就是wire型。而输出信号则由你自己来决定是寄存器输出还是组合逻辑输出，wire型、reg型都可以。always块下面进行赋值时，参数相当于被寄存器寄存，所以需要写reg型)

二、例化定义

这种情况是指，top层汇总各个模块时进行的例化，如上面的图就是system上需要例化PLL，time，water_led三个模块

针对这种情况应该这样定义：

首先top层的输入和输出依然按照上面的定义方法写

把不包含在输入输出里的参数（如上面的图就是clk_25m,locked,time_en）挑出来，写wire型

(原因：例化时，可以把里面的参数都当做信号线，只有连接作用)

三、仿真定义

这种情况是指在写TB文件的时候进行参数定义

首先说一下写TB文件的过程，TB文件可以测试小模块也可以测试top层，比如我要测试time模块，那么我就需要把time模块的输入输出复制到TB文件里面，也就是module后面的（....）；

复制过去按照例化的步骤进行例化

先将模块里面为input的变量写reg型，为output的变量写wire型

(注意不要忘写位宽，reg[x:x] wire[x:x])

但是有些时候会报错，所以按照上面的方法写完之后需要进行一些检测修改：

TB文件里面的参数在initial进行赋值的写reg型，其他的写wire型

然后就没有问题了

输入输出定义，例化定义，仿真定义已经几乎包含了FPGA里面所有的参数定义情况，按照我上面的方法定义不会有问题，下面来介绍一下原理：

Verilog 中变量的物理数据分为线型和寄存器型，线型数据包括wire,wand,wor等几种类型，其中wire最常用，对应于实际的数字电路，线型wire实际上就对应着硬件的连线，起到连接作用。寄存器是存储单元的抽象，寄存器数据类型的关键字是reg。常用来表示always模块内的指定信号，代表触发器。在always模块内被赋值的每一个信号都必须定义成reg型。

所以reg相当于存储单元，wire相当于物理连线

在需要赋值计算的时候就用reg，只是连接作用就用wire

将verilog中的wire和reg型数据相连接的方法

 blog.csdn.net/weixin_48843316/article/details/122025203

版权声明：本文为博主原创文章，遵循 [CC 4.0 BY-SA](#) 版权协议，转载请附上原文出处链接和本声明。

本文链接：https://blog.csdn.net/weixin_48843316/article/details/122025203

写过的典例：

数据说明：例化元件输出寄存器.dig；中间wire dig_temp;顶层程序输出寄存器dig。

目标：将例化元件的输出寄存器的值接到顶层程序输出寄存器中，因为两者都是寄存器，所以不能直接相连，需要借助中间一根线wire。

```
1. output reg [5:0] dig;  
  
2. wire [5:0] dig_temp;  
  
3. .dig(dig_temp);  
  
4. always@(dig_temp)  
  
5. begin dig=dig_temp; end
```

总结笔记：

reg：寄存器类型数据；wire：线类型数据

在使用例化模块进行编程时经常会遇到这两种类型的数据相连接的情况

wire型数据：在assign左侧被赋值；

reg型数据：在always@的左侧被赋值；

两者均能在assign和always@的右侧被使用。

Verilog 对assign和always的一点理解

 blog.csdn.net/iamoyjj/article/details/3478321

assign 用于描述组合逻辑

always@(敏感事件列表) 用于描述时序逻辑

敏感事件 上升沿 posedge，下降沿 negedge，或电平

敏感事件列表中可以包含多个敏感事件，但不可以同时包括电平敏感事件和边沿敏感事件，也不可以同时包括同一个信号的上升沿和下降沿，这两个事件可以合并为一个电平敏感事件。

在新的verilog2001中“，”和“or”都可以用来分割敏感事件了，可以用“*”代表所有输入信号，这可以防止遗漏。

合法的写法：

always@ *

always@ (posedge clk1,negedge clk2)

always@ (a or b)

`timescale 100ns/100ns //定义仿真基本周期为100ns

always #1 clk=~clk // #1代表一个仿真周期即100ns

所有的assign 和 always 块都是并行发生的！

并行块、顺序块

将要并行执行的语句写在

fork

//语句并行执行

join

将要顺序执行的语句写在

begin

//语句顺序执行

end

并行块和顺序块都可以写在

initial 或 always@ 之后，也就是说写在块中的语句是时序逻辑的

对assign之后不能加块，实现组合逻辑只能用逐句的使用

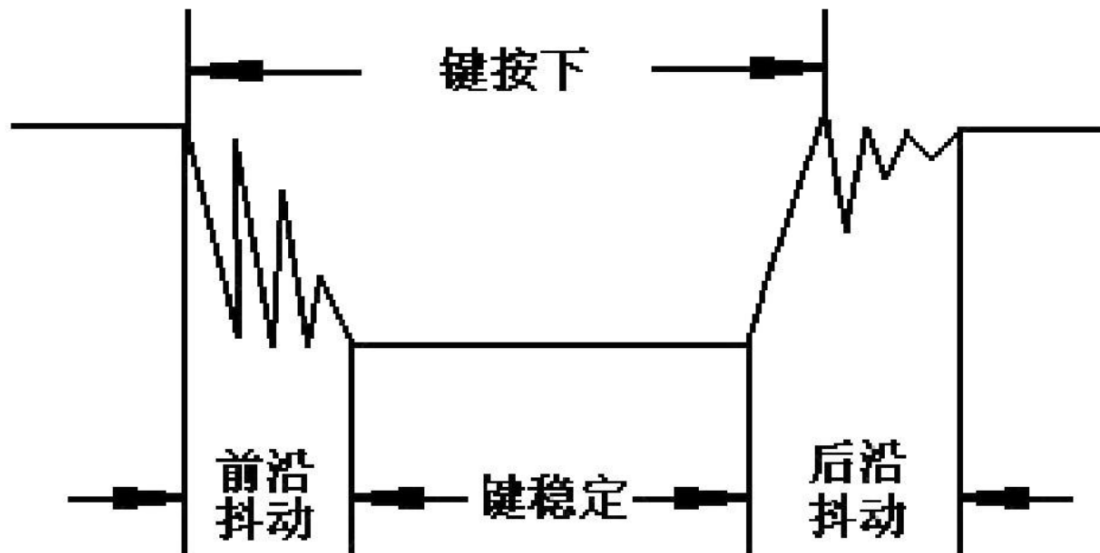
assign

组合逻辑如果不考虑门的延时的话当然可以理解为瞬时执行的，因此没有并行和顺序之分，并行和顺序是针对时序逻辑来说的。值得注意的是所有的时序块都是并行执行的。initial块只在信号进入模块后执行1次而always块是由敏感事件作为中断来触发执行的。

底层硬件底板按键抖动：关于按键抖动的时间、按键消抖

 blog.csdn.net/Eliauk1234/article/details/121381439

通常按键抖动所用的开关都是机械弹性开关，当机械触点断开、闭合时，由于机械触点的弹性作用，一个按键开关在闭合时不会马上就稳定的接通，在断开时也不会一下子彻底断开，而是在闭合和断开的瞬间伴随着一连串的抖动，如下图所示。



CSDN @程序员-虎哥

理想状态下的按键

理想按键波形

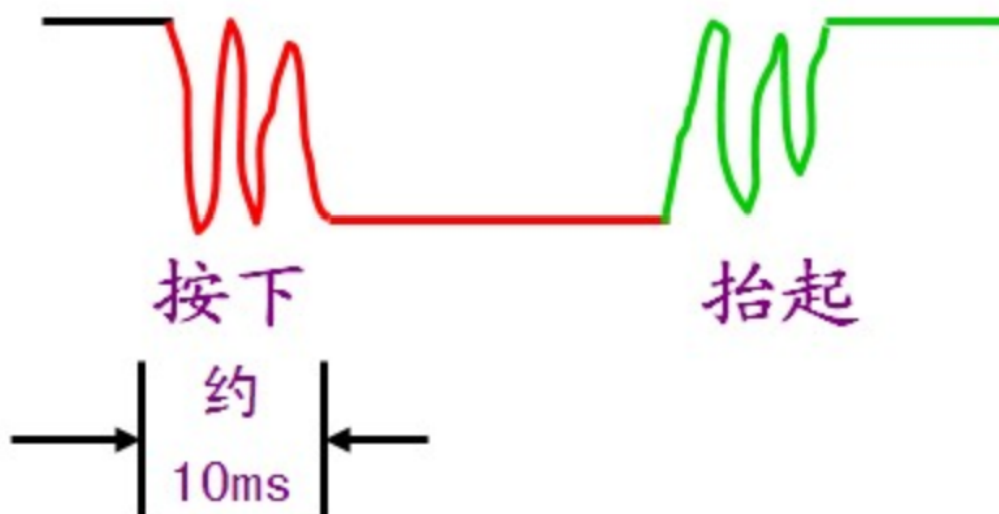


实际按键波形



CSDN @程序员-虎哥

按键稳定闭合时间长短是由操作人员决定的，通常都会在100ms以上，刻意快速按的话能达到40~50ms左右，很难在低了。抖动时间是由按键的机械特性决定的，一般都会在10ms以内，为了确保程序对按键的一次闭合或者一次断开只响应一次，必须进行按键的消抖处理。



CSDN @程序员-虎哥

在绝大多数情况下，是用软件即程序来实现消抖的。最简单的消抖原理，就是当检测到按键状态变化后，先等待一个10ms左右的延迟时间，让抖动消失后再进行一次按键状态检测，如果与刚才检测到的状态相同，就可以确认按键已经稳定的动作了。实际应用中，这种做法局限性大（实时性差）

软件去抖的几个思路：

1、延迟

先定义一个变量记录按键的状态：`char key`；

然后轮询检测按键状态，当按键状态改变的时候，判断为有按键动作，接下来进入延迟函数；等到延迟时间过了之后，再读取按键状态，如果按键状态仍为按下状态，则说明确实是有按键动作，不是抖动。

2、中断加延迟方式

单片机总是轮询状态会浪费很多资源，尤其是单片机运行的时间，可以通过中断方式来解决。中断模式下，单片机不需要轮询按键状态，当有中断产生的时候才会进入延时函数，进行按键去抖程序。

这种情况下，延时函数在哪里进行处理就有两种选择：

一种是放在中断函数中进行，这个时候中断函数占用的时间就比较长，影响响应速度；另一种是放在中断函数之外进行，这个时候就缩短了中断处理时间，但是这个时候就需要一个标志位来表明是否有中断产生，而且单片机也需要不断查询，只是节约了查询时候读取IO状态的步骤。

3、持续采样

持续采样会大大提高采样的准确度，但是同时也会增加CPU的开销。

在使用中，需要根据需要选择不同的采样频率，一般每10ms采集一次就足够了。

关于延迟方法：

简单的延时，可以采用空循环来实现，这个方法比较消耗CPU的资源，CPU任务较重的时候不建议使用。延时函数可以考虑使用定时器替换，但这又消耗了定时器资源，不过只要够用的话还是尽量用，毕竟可以减轻CPU的负担。

关于一次按键过程中产生多次中断的处理：

一次按键过程，可能产生多次中断，可以设置一个标志位来显示是否处于按键识别处理阶段，如果处于按键识别处理阶段即便产生中断也不进行任何响应，这样就可以忽略多余的中断了。

Verilog动态截取固定长度语法+:和-:



FPGA 专栏收录该内容

动态截取固定长度数据语法，即+:和-:的使用，这两个叫什么符号呢？[运算符](#)吗？

Verilog比较方便的一个特点就是数据的截取和拼接功能了，截取使用方括号[]，拼接使用大括号{}，例如

```
reg [7:0] vect;

wire a;
wire [3:0] b,
wire [5:0] c;

assign a = vect[1]; //取其中1Bit
assign b[3:0] = vect[7:4]; //截取4Bit
assign c[5:0] = {a, b[3:0], 1'b1}; //拼接
```

于是举一反三 (zi zuo cong ming)，为了实现动态截取固定长度数据的功能，使用软件编程的思维写了如下语句，功能很好理解，根据cnt的值，每次截取vect的5Bit数据。：

```
reg [7:0] vect;
reg [1:0] cnt;
wire [4:0] out;

assign out = vect[cnt+4:cnt];
```

一顿操作猛如虎，编译一看傻如狗。使用ModelSim编译之后，提示有如下语法错误：

```
** Error: test.v(10): Range must be bounded by constant expressions.
```

提示vect的范围必须为常量表达式。也就是必须为，vect[6:2]或vect[7:4]，不能是vect[a:0]，vect[4:b]，或vect[a:b]。额，这该怎么办呢？

既然有这个使用场景，那Verilog在设计之初就应该会考虑到这个应用吧！于是就去翻IEEE的Verilog标准文档，在5.2.1章节发现了一个用法可以实现我这个需求，那就是+:和-:符号，这个用法很少，在大部分关于FPGA和Verilog书籍中都没有提到。

5.2.1 Vector bit-select and part-select addressing

Bit-selects extract a particular bit from a vector net, vector **reg**, **integer**, or **time** variable, or parameter. The bit can be addressed using an expression. If the bit-select is out of the address bounds or the bit-select is **x** or **z**, then the value returned by the reference shall be **x**. A bit-select or part-select of a scalar, or of a variable or parameter of type **real** or **realtime**, shall be illegal.

Several contiguous bits in a vector net, vector **reg**, **integer**, or **time** variable, or parameter can be addressed and are known as *part-selects*. There are two types of part-selects, a constant part-select and an indexed part-select. A constant part-select of a vector reg or net is given with the following syntax:

```
vect [msb_expr:lsb_expr]
```

Both `msb_expr` and `lsb_expr` shall be constant integer expressions. The first expression has to address a more significant bit than the second expression.

An indexed part-select of a vector net, vector **reg**, **integer**, or **time** variable, or parameter is given with the following syntax:

```
reg [15:0] big_vect;  
reg [0:15] little_vect;  
  
big_vect [lsb_base_expr +: width_expr]  
little_vect [msb_base_expr +: width_expr]  
  
big_vect [msb_base_expr -: width_expr]  
little_vect [lsb_base_expr -: width_expr]
```

大致意思就是，可以实现**动态截取固定长度**的数据，基本语法为：

```
vect [base+:width] 或 [base-:width]
```

其中`base`可以为可变，`width`必须为常量。

下面来举几个例子来理解这个符号。

有如下定义：

```
reg [7:0] vect_1;  
reg [0:7] vect_2;  
  
wire [2:0] out;
```

以下写法分别表示什么呢？

```
vect_1 [4+:3];  
vect_1 [4-:3];  
vect_2 [4+:3];  
vect_2 [4-:3];
```

分为三步：

1.先看定义。

`vect_1[7:0]`定义是大端模式，则`vect_1[4+:3]`和`vect_1[4-:3]`转换后也一定为大端模式；`vect_2[0:7]`定义是小端模式，则`vect_2[4+:3]`和`vect_2[4-:3]`转换后也一定为小端模式。

2.再看升降序。

其中`+`表示升序，`-`表示降序

3.看宽度转换。

`vect_1[4+:3]`表示，起始位为4，宽度为3，**升序**，则`vect_1[4+:3] = vect_1[6:4]`
`vect_1[4-:3]`表示，起始位为4，宽度为3，**降序**，则`vect_1[4-:3] = vect_1[4:2]`

`vect_1[7:0]`

`vect_1[4+:3] = vect_1[6:4]`

7 6 5 4 3 2 1 0 起始4,宽度3,升序


`vect_1[4-:3] = vect_1[4:2]`

7 6 5 4 3 2 1 0 起始4,宽度3,降序


同理，

`vect_2[4+:3]`表示，起始位为4，宽度为3，升序，则`vect_2[4+:3] = vect_2[4:6]`
`vect_2[4-:3]`表示，起始位为4，宽度为3，降序，则`vect_2[4-:3] = vect_2[2:4]`

`vect_2[0:7]`

`vect_2[4+:3] = vect_2[4:6]`

0 1 2 3 **4** 5 6 7 起始4,宽度3,升序


`vect_2[4-:3] = vect_2[2:4]`

0 1 **2** 3 **4** 5 6 7 起始4,宽度3,降序


ModelSim仿真验证，新建test.v文件：

```
module test;

    reg [7:0] vect_1;
    reg [0:7] vect_2;

    initial
    begin
        vect_1 = 'b0101_1010;
        vect_2 = 'b0101_1010;

        $display("vect_1[7:0] = %b, vect_2[0:7] = %b", vect_1, vect_2);
        $display("vect_1[4+:3] = %b, vect_1[4-:3] = %b", vect_1[4+:3],
vect_1[4-:3]);
        $display("vect_2[4+:3] = %b, vect_2[4-:3] = %b", vect_2[4+:3],
vect_2[4-:3]);

        $stop;
    end

endmodule
```

在ModelSim命令窗口输入：

```
//进入到源文件所在文件夹
cd c:/users/whik/desktop/verilog
//编译
vlog test.v
//仿真
vsim work.
```

```
test
//运行
run -all
//运行结果
# vect_1[7:0] = 01011010, vect_2[0:7] = 01011010
# vect_1[4+:3] = 101, vect_1[4-:3] = 110
# vect_2[4+:3] = 101, vect_2[4-:3] = 011
# ** Note: $stop      : test.v(15)
#      Time: 0 ps  Iteration: 0  Instance: /test
# Break in Module test at test.v line 15
```

这种语法表示需要注意，前者起始位可以是变量，后者的**宽度必须是常量**，即vect[idx+:cnt]不符合语法标准，vect[idx+:4]或vect[idx-:4]才符合。

这个小语法知识点在大多数关于FPGA和Verilog的书籍中都没有，但是在IEEE Verilog的标准文档中有详细介绍，毕竟IEEE是标准的制定者。

Verilog 位拼接运算符{}语法要点总结_verilog {}_hanshuizhizi的博客-CSDN博客

 blog.csdn.net/hanshuizhizi/article/details/116521728

本文链接：<https://blog.csdn.net/hanshuizhizi/article/details/116521728>

Verilog位拼接运算符{}语法回顾

verilog中{}运算符用于“拼接”多个变量或者常量，基本用法如下：

1、变量的拼接

```
wire a[3:0], b[4:0];  
wire c[7:0];  
assign c = {a, b};
```

上面代码的执行结果如下图所示：

	7	6	5	4	3	2	1	0
a					4	3	2	1
b					4	3	2	1
c	4	3	2	1	4	3	2	1

2、变量与常量的拼接

```
wire b[4:0];  
wire c[7:0];  
assign c = {3'd5, b};
```

注意上面的常数5必须指明位宽，否则将出错。

例如，不能试图通过下面的语法把w初始化为8'b1111_1111，

```
wire w[7:0];  
assign w = {8{1}};
```

而应该写作：

```
wire w[7:0];  
assign w = {8{1'b1}};
```

3、变量或者常量的重复（扩展）与拼接

```
wire a[31:0], b[4:0];
wire c[7:0], d[7:0];
assign a = { 2{b}}, c, d };
assign c = {2{4'd5}};
assign d = {4{b}};
```

注意：如下写法的语法是错误的，2{b}必须用{}括起来再参与拼接。

```
assign a = { 2{b}, c, d };
```

4、用作字节序的交换

```
wire a[31:0], b[31:0];
assign a = {b[7:0], b[8:15], b[16:23], b[24:31]};
```

通常在大端模式和小端模式的处理器之间交互数据会需要字节序的交换。

5、位填充，把8位的有符号数扩展为32位

```
wire in[7:0];
wire out[31:0];
    assign out = { {24{in[7]}}, in };
```

要点总结

- 1、拼接中使用的常数建议必须指明位宽，否则可能出现问题。
- 2、拼接可以嵌套，但是嵌套的部分需要注意语法书写的细节（前述第三点）。
- 3、可以使用类似{n{reg}}的方式填充重复的reg，其中n为重复的次数。

verilog中for循环？

verilog中到底应不应该用for循环呢？我听我们老师说不让用for循环，要用计数器展开，但是我看的很多xilinx官方demo中for循环使用还是很频繁啊，像下面图片中alway块中，复位后竟然可以将存储器全部复位为0，i得变化不是根据时钟走吗？如果不是，那是根据什么变化？初入fpga得小白，求业界大佬帮忙解答下。不胜感激。

```
reg [word_width-1:0] reg_stages[stages_num-1:0];

always @(posedge clk or posedge rst)
begin:xhd10
    integer i;
    if(rst)
    begin
        for (i = 0; i <= stages_num - 1; i = i + 1)
        begin
            reg_stages[i]<=0;
        end
    end
    else
    begin
        if(ld)
        begin
            for (i = 0; i <= stages_num - 1; i = i + 1)
            reg_stages[i] <= (datain>>(i*word_width));
        end
        else if(oe)
        begin
            reg_stages[stages_num - 1]<=0;
            for (i = 0; i <= stages_num - 2; i = i + 1)
            reg_stages[i] <= reg_stages[i + 1];
        end
    end
end
```

[关注问题](#)[写回答](#)[邀请回答](#)[好问题 6](#)[1 条评论](#)[分享](#)[收起](#)

12 个回答

默认排序



硅农

订阅号 硅农/ASIC设计师/“云评测”数码博主/B站UP主

[+ 关注](#)

83 人赞同了该回答

Verilog设计中该不该用for循环？

先说结论，可以用，但是分时候，不会用的时候别乱用。

你老师说的计数器^Qxx，可能是让你写计数器的时候别用for循环，软硬件编程思维的不同。写一个计数器表示一段延时，软件编程的思维肯定的是用for循环，软件的执行是顺序执行，也就是一直处于这个循环中，直到不满足循环计数^Q条件而跳出循环。

而硬件中的计数器实现直接就是一组D触发器^Q，根据计数器的使能，加上在时钟边沿触发变化，再加上一个清零端（还可以加置位端）。软件和硬件这两个可以说，完全不是一个东西。

```
always @ (posedge clk or negedge rst_n) begin
    if(!rst_n)
        cnt <= 3'd0;
    else if(cnt_clear)
        cnt <= 3'd0;
    else if(cnt_en)
        cnt <= cnt + 1'd1;
end
```

如果我想实现软件思维的delay一段时间怎么做，直接用上面的计数器做一个flag



下载知乎客户端

与世界分享知识、经验和见解



相关问题

[verilog下实现大量数据的排序问题? 1 个回答](#)[verilog如何进行迭代运算? 1 个回答](#)[verilog代码怎么运行? 0 个回答](#)[verilog如何将各个模块连接起来? 4 个回答](#)


```
assign flag = cnt == 3'd3;
```

当cnt == 3 'd3时，我输出一个flag，表示我这段延时的标志。

i的变化不跟时钟走吗？

在Verilog中使用for循环的功能就是，把同一块电路复制多份，完全起不到计数的作用，所以这个i的意思是复制多少份你这段代码实现的电路，和时钟没有任何关系。主要是为了提高编码效率。

这个配图的功能就是把这块电路，复制stages_num份，然后每个clk下，执行的内容操作是相同的。所以能实现在复位时刻，把整个寄存器都赋值为0，然后复位释放执行其他操作。(这个图为题主提供)



题主提供图

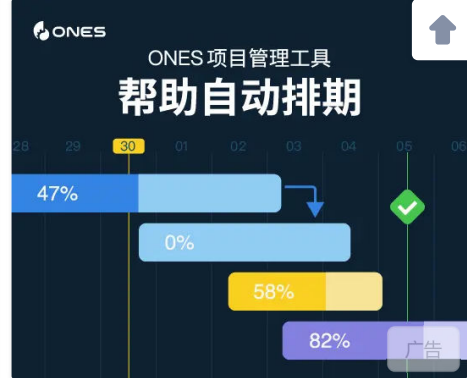
什么时候用

可以用，那么什么时候用，代码设计有规律的时候，可以配合generate for，合理使用 `generate+for` 循环可以提高编码效率，同样的赋值语句需要赋值多次。这个generate语句里面可以是任何有运算。

```
generate
  genvar i;
  for(i = 0;i < 16;i = i + 1)
    begin: neg_data
      assign neg_data_out[i*DATA_WIDTH+:DATA_WIDTH] =
        ~data_in[i*DATA_WIDTH+:DATA_WIDTH]
    end
endgenerate
```

同一个模块需要实例化多次也可以，比如这里有一个运算需要调用16个乘法器，如果把乘法器IP模块实例化16份，那岂不是要写很长的代码，所以用generate+for可以极大的提高编码效率。

```
generate
  genvar i;
  for(i = 0;i < 16;i = i + 1)
    begin: mult_12x12
      DW02_mult #(
```



帮助中心

知乎隐私保护指引 申请开通机构号 联系我们

举报中心

涉未成年举报 网络谣言举报 涉企虚假举报 更多

关于知乎

下载知乎 知乎招聘 知乎指南 知乎协议 更多

京 ICP 证 110745 号 · 京 ICP 备 13052560 号 - 1 ·
京公网安备 11010802020088 号 · 京网文
[2022]2674-081 号 · 药品医疗器械网络信息服务备
案 (京) 网药械信息备字 (2022) 第00334号 · 广
播电视节目制作经营许可证: (京) 字第06591号 ·
服务热线: 400-919-0001 · Investor Relations ·
© 2023 知乎 北京智者天下科技有限公司版权所有 ·
违法和不良信息举报: 010-82716601 · 举报邮箱:
jubao@zhihu.com



```

        .A_WIDTH(12),
        .B_WIDTH(12)
    ) u_DW02_mult0(
        .A(mult_a[i*12 +:12]),
        .B(mult_b[i*12 +:12]),
        .TC(1'b0),
        .PRODUCT(product[i*24 +:24])
    );
end
endgenerate

```

当然这样写debug会有一些困扰，不容易找到信号对应位宽数据。不过Verdi会显示每一个generate块，选中对应的块，加进去的波形就会是对应的bit信号。

最后就是做一个RAM/Regfile

```

reg      [DATA_WIDTH-1:0]  register[2**ADDR_WIDTH-1:0];
always @(posedge clk)begin
    if(!wren_n && !csen_n)
        register[addr] <= data_i;
end
always @(posedge clk)begin
    if(wren_n && !csen_n)
        data_o <= register[addr];
end

```

还有什么使用方法和问题欢迎评论区补充。

发布于 2021-01-04 22:04

▲ 赞同 83
▼
● 12 条评论
🚩 分享
★ 收藏
♥ 喜欢
收起 ^



Goblin

东南大学 集成电路硕士

+

关注

2 人赞同了该回答

forever

```

initial begin
    clk = 1;
    forever begin : clk_block
        #(clk_period/2) clk = ~clk;
    end
end

```

Note that a forever loop cannot be terminated via a disable statement.

在这里我们可以看出 verilog 中很多

05:08

Verilog中forever, repeat, while, for有什么不同

4937 播放 · 4 赞同

📺 视频回答

编辑于 2022-02-09 20:56 · 1223 次播放

▲ 赞同 2
▼
● 添加评论
🚩 分享
★ 收藏
♥ 喜欢

verilog中的for循环（以FIR分布式算法DA实现为例）

verilog中的for循环（以FIR分布式算法DA实现为例）

verilog中有几种循环语句，如for,while,repeat,forever等，for循环是唯一可综合的。在verilog中使用for循环，和在c语言中不一样，只需要记住一点，即verilog中for循环相当于**复制展开**，记住**复制展开**这个要点，就能很好理解复杂的代码。

先看下面这段代码，一时不好理解，抓住复制这点，重新写一下

```
always@(posedge clk or posedge reset)
begin:DA
    integer k,l;
    if(reset)
        begin
            for(k=0;k<=3;k=k+1)
                x[k]<=0;
            y<=0;
        end
    else
        begin
            for(l=0;l<=3;l=l+1) begin
                for(k=0;k<=1;k=k+1) begin
                    x[l][k]<=x[l][k+1];
                end
            end
            for(k=0;k<=3;k=k+1)
                begin
                    x[k][2]<=x_in[k];
                end
            y<=h[0]+(h[1]<<<1)+(h[2]<<<2)-(h[3]<<<3);
        end
    end

    genvar i;
    generate
        for(i=0;i<=3;i=i+1)
            case3s LC_Table0(.table_in(x[i]),.table_out(h[i])):
    endgenerate
```

我们按照for循环中的循环变量，将所有情况遍历并复制一遍，将各个循环变量的具体取值代入循环部分即可，代码如下

```
always@(posedge clk or posedge reset)
begin:DA
    integer k,l;
    if(reset)
        begin
//            for(k=0;k<=3;k=k+1)
//                x[0]<=0;
//                x[1]<=0;
//                x[2]<=0;
//                x[3]<=0;
//            y<=0;
//        end
    else
        begin
//            for(l=0;l<=3;l=l+1) begin
//                for(k=0;k<=1;k=k+1) begin
//                    x[l][k]<=x[l][k+1];
//                end
//            end
//            x[0][0]<=x[0][1];
//            x[0][1]<=x[0][2];
//
//            x[1][0]<=x[1][1];
//            x[1][1]<=x[1][2];
//
//            x[2][0]<=x[2][1];
//            x[2][1]<=x[2][2];
//
//            x[3][0]<=x[3][1];
//            x[3][1]<=x[3][2];
//
//            for(k=0;k<=3;k=k+1)
//                begin
//                    x[k][2]<=x_in[k];
//                end
//            x[0][2]<=x_in[0];
//            x[1][2]<=x_in[1];
//            x[2][2]<=x_in[2];
//            x[3][2]<=x_in[3];
//            y<=h[0]+(h[1]<<<1)+(h[2]<<<2)-(h[3]<<<3);
        end
end
end
```

```
// genvar i;
//generate
//      for(i=0;i<=3;i=i+1)
//          case3s LC_Table0(.table_in(x[i]),.table_out(h[i])):
//endgenerate

case3s LC_Table0(.table_in(x[0]),.table_out(h[0])):
case3s LC_Table0(.table_in(x[1]),.table_out(h[1])):
case3s LC_Table0(.table_in(x[2]),.table_out(h[2])):
case3s LC_Table0(.table_in(x[3]),.table_out(h[3])):
```

展开之后，这段代码的作用就比较明显了，4位输入信号x_in的每一位，在clk时钟节拍下，依次移位寄存，比如x_in[0]->x[0][2]->x[0][1]->x[0][0]，x_in[1]到x_in[3]类似。

可以看出，**复制次数为所有循环情况种类和**，如果两重循环，每重循环10种情况，就要复制100次，占用大量资源，这也是很多书让慎用for循环的原因。

如果补全后面一个查找表，这就是一个完整的FIR滤波器的分布式算法实现。

可以结合下面一张原理框图理解，完整代码引用书中有。

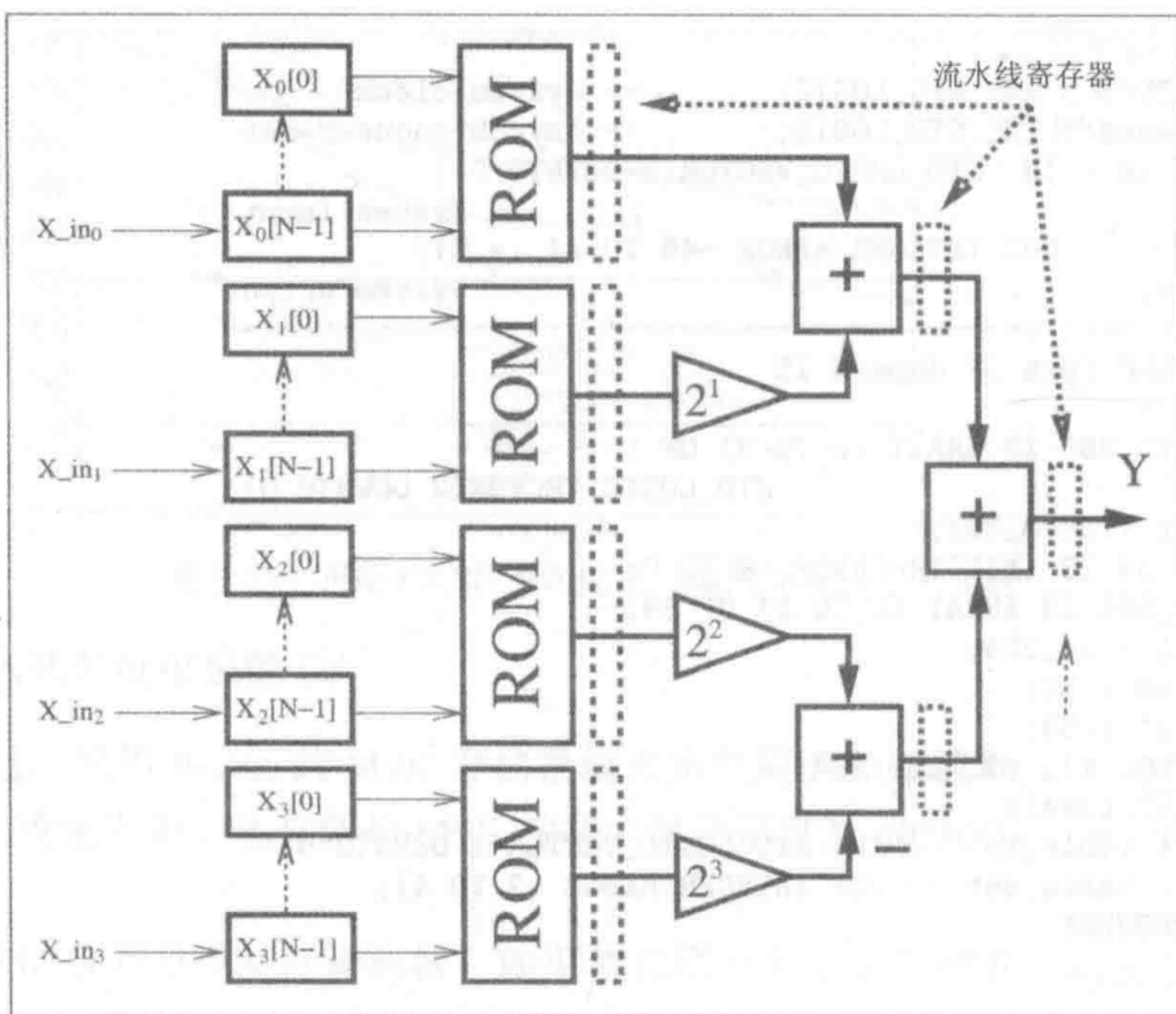


图 3-16 分布式算法 FIR 滤波器的并行实现 CSDN @leave_her_johnny

Verilog中对于for语句的使用

1. 在C语言或者其他编程语言中for语句往往用作循环语句，但是在硬件电路Verilog中一般不在可综合代码中使用，因为for循环会被综合器展开为所有变量情况的执行语句，每个变量独立占用寄存器资源，每条执行语句并不能有效地复用硬件逻辑资源，造成巨大的资源浪费。简单的说就是：for语句循环几次，就是将相同的电路复制几次，因此循环次数越多，占用面积越大，综合就越慢。

2. for语句的一般使用情况：

- 在testbench中使用，往往用于激励信号的生成。
- generate-for语句（可看上一篇博客）因为生成语句只要就是用于重复电路中所以可以使用简化代码。和generate-for语句本质上时相似的。
- 其他的有的情况可以使用 使用后也可以综合但不建议使用，若一些简单的赋值重复很多可以使用来简化代码。

3. 语法：

- for语句不单独使用，一般放在always块里
- 循环变量i 一般定义为integer型一般放在always块内定义
- always块应有名字 和generate相似

4. 举个栗子：

```
always @(posedge clk)
begin
if(!rst_n)
num = 0;
else
begin
for(i=0;i<13;i=i+1)
if(data[i]) num = num + 1;
end
end
```

//这个for语句里用的是阻塞赋值，在综合时会将for语句展开综合成一长串的十三个类似的电路，并且是在一个时钟周期内完成计算，但是这样做可能会导致时序冲突即若频率很高一个周期完成不了从而产生错误，在对时钟周期没有严格要求的情况下一般不这样使用。

```
always @(posedge clk)
begin
if(!rst_n)
num <= 0;
else
begin
for(i=0;i<13;i=i+1)
if(data[i]) num <= num + 1;
```

end

end

//这里是非阻塞赋值，在综合时会把for语句展开综合成十三个类似的并行电路，相当于十三句非阻塞赋值给一个变量赋值，这样会导致一个时钟周期只能计数一次且每次都是最后一位的赋值状态。不能完成计数。

所以综上，for语句是将电路进行复制，其运行状态还取决于内部是阻塞赋值还是非阻塞赋值。在可综合程序中一般不推荐使用。

verilog for循环那些事

verilog中的for循环，是一个头疼问题，传统的generate for循环，一定是可综合的，但是不够灵活，有些场景下实现不了想要的一些功能（尤其是参数化时候），此帖记录一下个人经验。

1，generate 用法（for，if，case）

贴一个链接，讲的很详细。补充一点就是，generate for循环体内，需要用assign或always（理解为一个新的verilog code，那自然按照verilog语法，要么assign，要么always，不能直接写赋值）

[verilog语法之generate语句的基本认识_小蒋啊的博客-CSDN博客_generate语句](#)

2，非generate的for用法

目前理解是，SV支持这些写法，EDA工具基本都已支持其编译，仿真及综合。

①always中for循环

做一些merge的时候比较常用，此时需要在每次循环开始对data赋初值。比如这里merge处置设为0，否则每拍最开始merge会使用上一拍的结果参与到merge。

此时的for循环中不能再例化module

```
reg a;
reg b;
reg merge
always @* begin    // verdi中对这个always块会显示unname,内部的for循环的name倒是依然在
    merge = 0;
    for (int/integer i;i< XX;i=i+1) begin: FOR1    // always块内的循环，使用int、integer定义i
        a[i] = sss;
        if()begin
            b[i]=kkk;    // always中不管再出现什么，都可以直接赋值了，不能用assign，也不必再什么其他块写法
            merge = merge | b[i];    // 看着像组合逻辑环error，但是还真允许
        end
    else.....
end
end
```

②for循环中的always


```
for (genvar i;i<XX;i=i+1) begin:FOR2    //最外面的for循环，使用genvar来定义i，不然编译报sv语法错误
    always @* begin
        .....
    end
end
```

③直接for循环

```
for (genvar i;i<XX;i=i+1) begin:FOR2    //最外面的for循环，使用genvar来定义i，不然编译报sv语法错误
    assign .....; //将条件判断语句都换成 ? :，则可以全写成assign形式
end
```

3，扩展

在for循环中，常用到位宽选择：

合法：`a[i*4+:4]` //例化的电路中，每个选择a的某一段4bit，位宽是确定的

不合法：`a[i:0]` // for循环中，i一直在变，则对于每个例化出来的电路而言，选中的a的field位宽不确定，lint error（经过尝试，改写成`a[0+:i]`却是ok的，很怪）

verilog中的3种for循环的写法



verilog 专栏收录该内容

直接进入正题，对于for循环的三种循环做以下介绍：

1. for循环用于例化多个模块

```
genvar i;
generate
    for(i=0;i<12;i=i+1)begin:portx_input_path_tdest
        input_path_tdest portx_input_path_tdest (
            .switch_clk (/s_axi_aclk*/switch_clk),
            .switch_rstn (/switch_and_fifo_aresetn*/switch_rstn)
        );
    endgenerate
```

注意点：

- (1) 例化模块在for循环内部
- (2) for循环需加名字
- (3) `genvar` 定义在generate外面，可以全局使用（不推荐使用，最好一个generate for，对应一个循环变量）

2. for循环写多个always块或者写多个assign语句4

```

genvar i;
generate
    for(i=0;i<12;i=i+1)begin:portx_input_path_tdest
        always @(posedge clk or posedge rst) begin
            if (rst) begin
                // reset

            end
            else if () begin

            end
        end
    end
endgenerate

```

注意点：

- (1) always块在for循环内部
- (2) for循环需加名字
- (3) genvar 定义在generate外面，可以全局使用（不推荐使用，最好一个generate for，对应一个循环变量）

其实这个for循环类似第一个。

3. for循环在一个always块中对写多条类似语句

```

always@(posedge clk or posedge rst) begin:exsit_frame
    integer m;
    if(rst)begin
        transmit_permit <= 1'b0;
        MarkAllFramesRed <= 128'd0;
        for (m=0;m<32;m=m+1)begin
            current_token_Tc[m] <= MAX_CURRENT_TOKEN;
            current_token_Te[m] <= MAX_CURRENT_TOKEN;
        end
    end
end

```

注意点：

- (1) 循环变量类型为integer

(2) `always`块需要加名字，`for`不用加名字。

小贴士：

`generate for`中支持`data[3i+8 : 3i]`，但是单纯`for`循环，即第三种不支持这种写法，可改为`data[3i+ : 8]`。

< RTL coding的艺术 > verilog中for循环中循环变量int/genvar区别讲解

目录

一、问题背景

二、正确的代码展示

三、错误的循环变量类型报错

四、总结

一、问题背景

今天被for循环的循环变量类型搞疯了，原来放在不同位置、起不同作用的for循环，必须对应特定的循环变量类型，否则将会报编译错误，本篇将讲清楚它们的来龙去脉。

二、正确的代码展示

先看两种for循环使用情况：

```

30 );
31 generate
32     localparam int unsigned ADDER_NUM = SUM_WIDTH/ADD_WIDTH;
33
34     logic [ADDER_NUM-1:0] carry_in;
35     logic [ADDER_NUM-1:0] carry_out_o;
36     logic [ADDER_NUM-1:0][ADD_WIDTH-1:0] add_temp0,add_temp1, add_out_o;
37     assign add_temp0 = add0_i;
38     assign add_temp1 = add1_i;
39     assign out_o      = add_out_o;
40
41     for (genvar i = 0; i < ADDER_NUM; i++) begin
42         assign {carry_out_o[i],add_out_o[i]} = add_temp0[i] + add_temp1[i] + carry_in[i];
43     end
44
45
46     if(MODE_NUM == 4)begin
47         always_comb begin
48             carry_in = 'b0;
49             case(precision_i)
50                 0:begin
51                     carry_in      = 'b0;
52                 end
53                 2:begin
54                     for (int i = 0; i < ADDER_NUM; i=i+2) begin
55                         carry_in[i]      = 1'b0;
56                         carry_in[i+1]      = carry_out_o[i];
57                     end
58                 end
59                 4:begin
60                     for (int i = 0; i < ADDER_NUM; i=i+4) begin
61                         carry_in[i]      = 1'b0;
62                         carry_in[(i+1) +:3] = carry_out_o[i+:3];
63                     end
64                 end
65                 default: begin
66                     for (int i = 0; i < ADDER_NUM; i=i+8) begin
67                         carry_in[i]      = 1'b0;
68                         carry_in[(i+1) +:7] = carry_out_o[i+:7];
69                     end

```

三、错误的循环变量类型报错

上面两个for循环中，一个以genvar为循环变量，另一个以int为循环变量，能不能交换呢？

答案是不能，交换后VCS编译会报systemverilog语法错误。

```

Error-[SE] Syntax error
Following verilog source has syntax error :
...", 41:
token is 'int'
  for (int i = 0; i < ADDER_NUM; i++) begin
      ^
SystemVerilog keyword 'int' is not expected to be used in this context.
1 error
CPU time: .274 seconds to compile
/home/users/shawn lee/HPV/ project/vfu%(wycmn04-in)

```

实际上两者区别如下：

1、genvar循环用于产生多套电路，各套电路之间**必须独立**；int循环可以用于同一个逻辑的**累积赋值**，例如累加，但是也可以用于**多套独立组合逻辑**描述。

2、genvar循环**一定在过程块之外**（always，initial），int循环**一定在过程块内部**。

四、总结

两种for循环都使电路复制变得简单，但是要记住他们的使用场景。

1、genvar循环用于产生电路：从无到有，类似于描述N个孕妈生孩子的过程。

2、int循环用于描述行为，只能在**已有的**对象上进行，类似于描述孕妈们生下的孩子们交朋友的过程。

verilog语法之generate语句的基本认识



fpga 专栏收录该内容

最近在至芯科技的书上看到的，觉得还行吧，给大家分享一下。

一、为什么学习generate？

在设计中，很多情况下需要编写很多结构相同但是参数不同的赋值语句或者逻辑语句，如果在参数量很大的的情况下，原本的列举就会显得心有余而力不足。c语言中常用for语句来解决此类问题，[verilog](#)则为我们提供了generate语句。

二、generate的基本概念及语法

generate语句的最主要功能就是对module、reg、assign、always、task等语句或者模块进行复制。

generate语句有generate_for、generate_if、generate_case三种语句。

下面开来看看这三种语句有什么不同？

1、generate_for语句

注：（1）、必须使用genvar申明一个正整数变量，用作for循环的判断。（genvar是generate语句中的一种变量类型，用在generate_for中声明正整数变量。）

（2）、需要复制的语句必须写到begin_end语句里面。就算只有一句！！！！！！

（3）、begin_end需要有一个类似于模块名的名字。

例1、利用generate_for来复制assign语句

```
module generate_for(
    input  [7:0]          data_in
    output [1:0]          t0
    ,
    output [1:0]          t1
    ,
    output [1:0]          t2
    ,
    output [1:0]          t3
);

wire [1:0] temp [3:0]; //定义位宽为2，深度为4的temp

genvar i; //利用genvar
声明正整数变量
generate for(i=0;i<4;i=i+1) //复制模块
begin : gfor //begi_end的名字
    assign temp[i] = data_in[2*i+1:2*i];
end
endgenerate

assign t0 = temp[0]; //assign temp[0] = data_in[1:0];
assign t1 = temp[1]; //assign temp[1] = data_in[3:2];
assign t2 = temp[2]; //assign temp[2] = data_in[5:4];
assign t3 = temp[3]; //assign temp[3] = data_in[7:6];

endmodule
```

```
`timescale 1ns/1ps

module tb;
reg [7:0] data_in;
wire [1:0] t0;
wire [1:0] t1;
wire [1:0] t2;
wire [1:0] t3;

initial begin
    data_in=8'd0;
    #200;
    data_in = 8'b1100_0011;
end
```


		Mega													
	data_n	00111100	00000000					11000011					00111100		
	t0		00				11					00			
	t1		11									11			
	t2		11									11			
	t3		00					11				00			

2、generate_if语句

generate_for用于复制模块，而generate_if则是根据模块的参数（必须是常量）作为条件判断，来产生满足条件的电路。相当于判断语句。

例2、

```

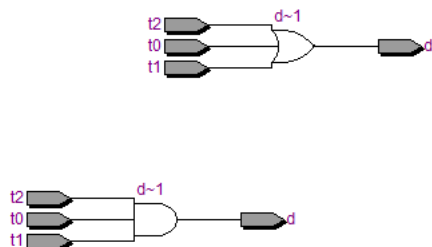
module generate_if(
    input t0
    ,
    input t1
    ,
    input t2
    ,
    output d
);

localparam S = 6; //定义模块所需参数,用于判断产生电路

generate
    if(S < 7)
        assign d = t0 | t1 | t2;
    else
        assign d = t0 & t1 & t2;
endgenerate

endmodule

```



由上面我们可以看出模块参数不同，选择的电路就不相同。

3、generate_case语句

generate_case其实跟generate_if一样，都是根据参数（都必须为常量）作为判断条件，来产生满足条件的电路，不同于使用了case语法而已。

例、3

```

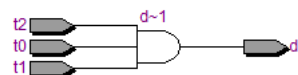
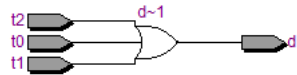
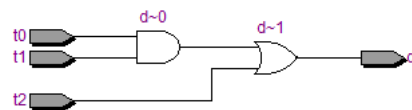
module generate_case(
    input t0
    ,
    input t1
    ,
    input t2
    ,
    output d
);

localparam S = 8; //定义模块所需参数,用于判断产生电路

generate
    case(S)
    0:
        assign d = t0 | t1 | t2;
    1:
        assign d = t0 & t1 & t2;
    default:
        assign d = t0 & t1 | t2;
    endcase
endgenerate

endmodule

```



由上面3副图其实就可以看出generate_if和generate_case功能其实是一样的。但是比较常用的就是generat_for语句。希望大家能好好学一学，毕竟generate语句还是很有用的噢~
未完待续！！！！！！

关于verilog中是把output设成reg变量和内部设置一个reg变量的讨论

为了能把一个输出信号赋给输出端口，常看到如下的两种处理方式。

方式A：

```
1. module test1(clk,counter);  
2.  
3. input clk  
4.  
5. output[7:0] counter  
6.  
7. reg[7:0] counter_reg ;  
8.  
9. always@(posedge clk)  
10.  
11. begin  
12.  
13. counter_reg<=counter_reg+1  
14.  
15. end  
16.  
17. assign counter=counter_reg;  
18.  
19. endmodule
```



方式B：

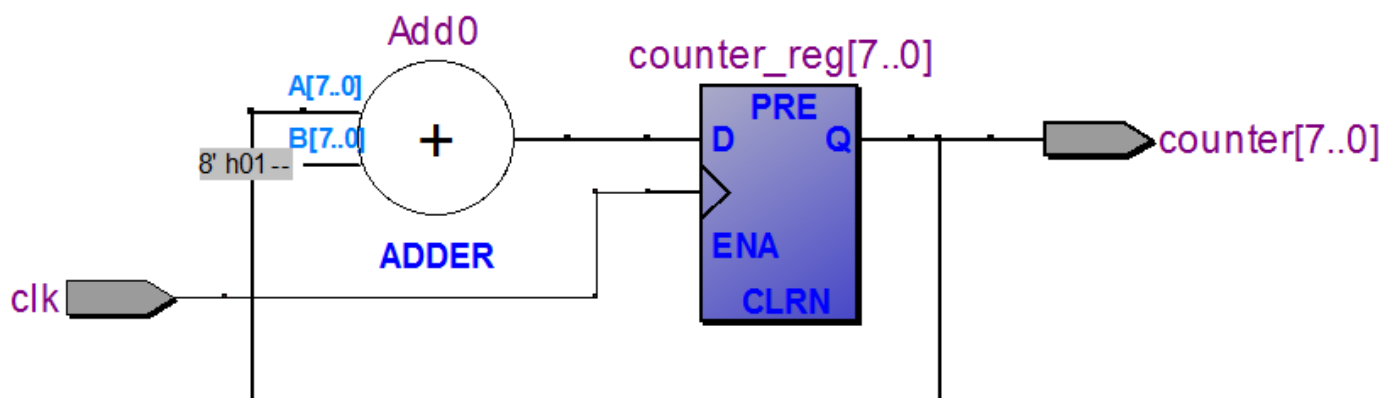
```
1. module test1(clk,counter);  
2.  
3. input clk  
4.  
5. output reg[7:0] counter ;  
6.  
7. always@(posedge clk)  
8.  
9. begin  
10.  
11. counter<=counter+1'b1 ;  
12. end  
13. endmodule  
14.  
15.
```

A是单独设置内部变量，在always中将输出信号赋给counter_reg，让后通过assign将counter和counter_reg连接起来。

B是直接将输出端口设置成reg型信号，直接在always中赋值。

综合的结果表明，两种处理方式的结果一致。

综合后的RTL级视图均为：



R：关于总线 Bus 的使用体会

有两种方式：

1. 利用总线中的每一位的电平值，访问时加下标访问

e.g., Bus[0]

常用于标志位等的设置

2. 利用总线某几位的数值大小，访问时作为一个数进行访问

e.g., Bus[7:4] 的二进制数值大小代表某个含义，而 Bus[3:0] 的大小可能又代表另外一个含义。这种使用方式，实际上类似数组，只不过为了适应硬件电路，单根线只有二值电平的情况，将每个十进制值，用四位（用的最多）表示出来